

Name: Mary-Rose Tracy

ID#: 1001852753

Date: 11/11/2024

Instruction

What the question is

MY NOTES

Decidability

TM Closure: 3.16d, 3.16b (Use Nondeterminism), 3.16c (Use Nondeterminism)

3.16 Show that the collection of Turing-recognizable languages is closed under the operation of

a. union.

b. concatenation.

c. star.

d. intersection.

e. complement.

3.168] so let's say X & Y be 2 Turing recognizable lang's that have T.M M_X & M_Y respectively. Now concat of these lang's is denoted by L_{XY} & the T.M recognizing this lang is M_{XY} .

Let S be a word for L_{XY} , M_{XY} works for an input str. S as shown

- it (Q) (divides) each str of XY into S_1 & S_2 non-deterministically.
- It runs...
 - S_1 to M_X , if M_X halts & reject, reject
 - S_2 to M_Y , if M_Y accepts, accepts. If M_Y halts & rejects, reject.

Thus, it can be said that collection of Turing recognizable lang's is closed under concatenation operation.

3.16 6. Closure under concat

forgot to say copy s_1 to tape 2 then run M_1 & copy s_2 to tape 2 then run M_2 . ~~for the previous answers~~ to the 3 questions

consider 2 Turing recog Lang's L_1 & L_2 recognized by T.M. M_1 & M_2 respectively. We need to construct a Turing machine that recognizes the lang $L_1 \cup L_2 = \{XY \mid X \in L_1 \text{ & } Y \in L_2\}$.
Construct using non-determinism.

1. Let M be a non-deterministic T.M. that will decide if an input w belongs to $L_1 \cup L_2$.
2. M will split the input w non-deterministically into two parts s : $w = XY$.
3. M will then simulate M_1 on X & M_2 on Y .
4. If both simulations accept, then M accepts w . Otherwise reject s .

Since M_1 & M_2 are Turing. M that recognize L_1 & L_2 respectively, we can construct a T.M. M that recognizes $L_1 \cup L_2$ by using non-determinism to guess the split & simulate M_1 & M_2 on each pt. therefore $L_1 \cup L_2$ is Turing recognizable.

3.16C | Let's say X be a Turing recognizable lang
& the T.M is M_X .
Now X^* is lang obtained from star operation
on X .

The T.M for this lang be M_{X^*} .

M_{X^*} is ...:

For ...

On an input str s of X , it non-deterministically
C/D divided the str. into $s_1, s_2, \dots, s_n$.
For each of those C/D divided parts M_X runs,
suppose, M_X all divided parts then s is
accepted by M_X else s is rejected by M_X .

Thus, it can be said that collection of Turing
recognizable lang's is closed under
star operation.

3.16 Closure under Kleene

To show that Turing-recognizable Langs are closed under star operation, let L be a Turing-recognizable Lang. recognized by a T.M. M . We need to construct a T.M. that recognizes $L^* = \{ w_1 w_2 \dots w_k \mid k \geq 0 \text{ \& } w_i \in L \text{ for each } i \}$ construct using nondeterminism:

1. Let M^* be a nondeterministic T.M. that will decide if an input w belongs to L^* .
2. M^* will nondeterministically split w into substrings $w_1, w_2, \dots, w_k$.
3. For each sub str w_i , M^* will simulate M on w_i .
4. If each simulation of M accepts for substrings $w_1, w_2, \dots, w_k$, then M^* accepts w .
5. If any simulation does not accept, then M^* rejects w .

Since M recognizes L , the T.M. M^* can recognize L^* by nondeterministically guessing the substrings & verifying each w/ M . Thus, L^* is Turing recognizable.

3.16 D Let's say X, Y be two Turing recognizable Langs that have T.M. M_X & M_Y respectively. Now the intersect of these Lang's is denoted by L_{XY} & the T.M recognizing this Lang's is M_{XY} .

For input str s from L_{XY} , M_{XY} works for as shown...

T.M M_X runs on s . If it accepts s then M_Y runs on s . Else s is rejected.

suppose, M_X accepts s then it's accepted by the T.M otherwise s is rejected.

Thus, it can be said that the collection of Turing recognizable Lang's is closed under intersect operation.

3.16 d Closure under intersect

To show that Turing-recognizable langs are closed under intersection, let L_1 & L_2 be Turing recognizable langs recognized by T.M. M_1 & M_2 respectively. We want to construct a Turing M that recognizes $L_1 \cap L_2 = \{w \mid w \in L_1 \& w \in L_2\}$.

Construction

1. Construct a T.M. M that, on input w , simulates both M_1 & M_2 on w simultaneously (this can be done by alternating steps between the 2 machines).
2. If both M_1 & M_2 accept w , then M accepts w .
3. If either M_1 or M_2 rejects w (or loops forever w/out accepting), M does not accept w .

Since M_1 & M_2 are T.M.'s that recognize L_1 & L_2 , we can construct M to recognize $L_1 \cap L_2$ by simulating both machines & accepting only if both accept. Therefore, $L_1 \cap L_2$ is Turing-recognizable.

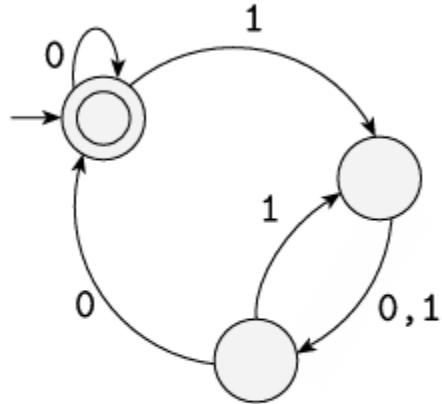
Concat & star use nondeterminism by guessing splits in the input & verifying each part

Intersect - uses simultaneous simulation to check acceptable by both Turing machines.

3.16 Redo!! To show that the Class of Turing
recognizable Lang's is closed under intersection
for Languages L_1 & L_2 recognized by
T.M's M_1 & M_2 :

1. Construct a nondeter T.M M that will decide
if an input w belongs to $L_1 \cap L_2$.
2. M will nondeterministically guess which machine
 M_1 or M_2 to run first on input w .
3. M will run M_1 on w in one possible branch
of computation, & M_2 in another.
4. If w is accepted by both M_1 & M_2 in any
branch, then M accepts w .
5. If any branch finds that w is Not accepted
by either M_1 or M_2 , it rejects that branch
of computation.
by using nondeterminism, we allow M to
verify that w belongs to both L_1 & L_2
by exploring all possible branches
of computation that involve M_1 & M_2
running on w . If any branch leads
acceptance by both M_1 & M_2 , M accepts;
otherwise it rejects.

^A4.1 Answer all parts for the following DFA M and give reasons for your answers.



- | | |
|---|---|
| a. Is $\langle M, 0100 \rangle \in A_{\text{DFA}}?$ | d. Is $\langle M, 0100 \rangle \in A_{\text{REX}}?$ |
| b. Is $\langle M, 011 \rangle \in A_{\text{DFA}}?$ | e. Is $\langle M \rangle \in E_{\text{DFA}}?$ |
| c. Is $\langle M \rangle \in A_{\text{DFA}}?$ | f. Is $\langle M, M \rangle \in EQ_{\text{DFA}}?$ |

4.1 A) $w \in \text{on input } \langle M, w \rangle$, where M is a DFA & w is str.
 starting & final (accepting) state of M is x . After taking the
 1st input 0 from str 0100 to the next state still x . Now 1 input
 to str 0100 the next state = y . Sim taking 4th '0' input
 0100 the next state becomes x which is accepted
 M does Accept input str 0100.

B) After taking the 1st input 0 from str 011, the next state
 is still x .
 After taking 2nd input 1 from str 011 the next state is y .
 Similarly after taking 3rd '0' input from 011 the next
 state becomes z which isn't accepted

M Does NOT accept input str 011

C) Input str w is not given in Aqfa. Here Aqfa is
 not in proper format.

M Does NOT accept Input str.

D) Consider a T.M w which decides whether reg
 expression can generate provided str or not
 $w = \text{on input } \langle R, w \rangle$, where R is a reg expression, w is str.
 Here, $P \neq M$ is a DFA instead of regular expression but for
 AReg it should be reg expression.

M cannot generate str 0100

E) ~~E~~ Do T.M which decides E_{DFA} . $w = \text{on } \langle M \rangle$, where M is DFA
 mark init state of M , that is x ,
 repeat state till new state not marked & found init &
 end state are the same that is x . So when
 applying 1st step of DFA theorem then init as well as empty
 as final state both get marked. It is not acceptable when
 final state is marked. M cannot accept any str. M is not N

F) T.M w which decides EQ_{DFA} . Here $L(M) = L(M)$
 therefore deriv lang $L(L)$ becomes empty. Thus T.M
 w accepts deriv lang $L(L)$.

yes the Lang of M Accepts itself.

4.1 | A) For all questions, A_{DFA} is a lang which test whether given string is accepted provided str. w or not. Use the following steps to check DFA accepted given str. or not.

DFA M

Consider a T.M. M which decides A_{DFA} . M on input $\langle M, w \rangle$, where M is a DFA & w is str.

1. on w input simulate M .
2. when simulation ends at accept states then it's accepted. when simulation doesn't end at accept st. $\rightarrow$ rejected

M accepts str 0100 or NOT. Above figure the starting & final (accepting) state of M is x . After taking the 1st input 0 from str 0100. the next state is still x . After taking the second input 1 from str 0100 the next state is y . Similarly after taking fourth '0' input from 0100 the next state becomes $x \rightarrow$ acceptable state. M accepts input state 0100.

B) 011

As shown in above fig the starting & final (accepting) state of M is x . After taking the 1st input 0 from str 011. the next state is still x . After taking the 2nd str input 1 from str 011 the next state is y . Similarly after taking 3rd '0' input from 011 the next state becomes z which is not an acceptable state.

Does NOT

Thus M Does NOT accept input string 011

D) M, $P \neq$

Use above theorem to check whether M accepts given string or not. Here input str. or NOT. Here input str. w isn't given A_{DFA} . Here A_{DFA} is not proper format.

Does NOT

Thus, H Does NOT accept input string

4.1] A) For all questions, A_{DFA} is a lang which test whether given str. is accepted provided str. w or not. Use the following steps to check DFA accepted given str. or not.

Consider a T.M. M which decides A_{DFA} . M on input $\langle M, w \rangle$, where M is a DFA & w is str.

1. on w input simulate M .
2. when simulation ends at accept states then it's accepted. when simulation doesn't end at accept st. $\rightarrow$ rejected

M accepts str 0100 or NOT. Above figure the starting & final (accepting) state of M is x . After taking the 1st input 0 from str 0100. the next state is still x . After taking the second input 1 from str 0100 the next state is y . Similarly after taking fourth '0' input from 0100 the next state becomes $x \rightarrow$ acceptable state. M accepts input state 0100.

B) 011

As shown in above fig the starting & final (accepting) state of M is x . After taking the 1st input 0 from str 011. the next state is still x . After taking the 2nd str input 1 from str 011 the next state is y . Similarly after taking 3rd '0' input from 011 the next state becomes z which is not an acceptable state.

Does NOT

Thus M Does NOT accept input string 011

C) M, P, T

Use above theorem to check whether M accepts given string or not. Here input str. or NOT. Here input str. w isn't given A_{DFA} . Here A_{DFA} is not proper format.

Does NOT

thus, H Does NOT accept input string.

4.2 Consider the problem of determining whether a DFA and a regular expression are equivalent. Express this problem as a language and show that it is decidable.

4.2 Express Lang $L = \{ \langle R, S \rangle \mid R \text{ is a DFA, } S \text{ is reg expression} \mid L(R) = L(S) \}$. RT decides the lang $EQ_{DFA} = \{ \langle P, Q \rangle \mid P \text{ \& } Q \text{ are DFA } L(P) = L(Q) \}$.

Assume $T = T.M$ which decides Lang L .

Can be:

T on input $\langle R, S \rangle$ $R = \text{DFA}$ $S = \text{reg expression}$.

Convert R into DFA D_R using algorithm in proof of Kleene Theorem.

Operate $T.M$ as a decider F using theorem 4.5 on input $\langle R, D_S \rangle$

IF F accepts, accept the lang L .

$- F$ rejects, reject the lang L .

4.2 Proof of the decidability of the Lang

- Express the lang as $L = \{ \langle R, S \rangle \mid R \text{ is a deterministic Finite Automata (DFA) \& } S \text{ is expression w/ } L(R) = L(S) \}$.
- Recollect the Theorem 4.5 states a T.M. T that decides the lang $EQ_{DFA} = \{ \langle P, Q \rangle \mid P \& Q \text{ are DFA } L(P) = L(Q) \}$.
↑
deterministic Finite Automata
- Assume that T is the T.M. which decides lang L . it can be...
 $T =$ on input $L \in \{ \langle R, S \rangle \}$, where R is a DFA & S is reg. expression:
convert R into a DFA D_R using the algorithm in the proof of Kleene's Theorem.
Operate a T.M. as a decider F using Theorem 4.5 on input $\langle R, D_R \rangle$
if F accepts, accept lang L .
if F rejects, reject lang L .

4.3 Let $ALL_{DFA} = \{ \langle A \rangle \mid A \text{ is a DFA and } L(A) = \Sigma^* \}$. Show that ALL_{DFA} is decidable.

4.3] We need to show that there is a T.M. that can decide whether a given DFA A accepts all possible strings over its alphabet Σ ($L(A) = \Sigma^*$).
input: input to the decider is a DFA A .

2. Reachable States:

Perform BFS Breadth 1st Search, or DFS starting from initial state of A .

Mark all states that are reachable from s_0 or s_1 .
If any state is not reachable, then A does not accept all strings. In this case, reject. ($L(A) \neq \Sigma^*$)

3. Check all reachable states are accepting.
After identifying the reachable states, check if all these states are accepting states.
If any reachable state is a non-accepting state, then A does not accept all strings in this case, reject the input.

4. Accept if All reachable states are Accepting:
If all reachable states are accepting states then A accepts all strings, & $L(A) = \Sigma^*$. In this case, accept input.

The ALG decides whether $L(A) = \Sigma^*$ for a given DFA A . Since the procedure always halts ALL DFA is decidable.

4.4 Let $A \in \text{CFG} = \{ \langle G \rangle \mid G \text{ is a CFG that generates } \epsilon \}$. Show that $A \in \text{CFG}$ is decidable.

4.4] Decidability of $\text{Lang } A_{\text{ECFG}} = \{ \langle G, \epsilon \rangle \mid G \text{ is a CFG that } \epsilon \in L(G) \}$

Given: In this $\text{Lang } A_{\text{ECFG}}$ is given

$\text{Lang } A_{\text{ECFG}}$ is decidable, build a T.M T for deciding the $\text{Lang } A_{\text{ECFG}}$ For all $\text{CFG} = G$.

is The Gram:

• G deriv ϵ then $T(\langle G, \epsilon \rangle)$ accepts

• G Doesn't deriv ϵ then $T(\langle G, \epsilon \rangle)$ rejects.

For proving the decidability of A_{ECFG} Firstly convert the $\text{CFG} = G$ into equiv G' in CNF. If $S \rightarrow \epsilon$ is the rule in the $\text{CFG } G'$ then it means that G' deriv ϵ . If the $\text{CFG } G'$ deriv ϵ then G also deriv it as $L(G) = L(G')$. As G' is in CNF only possible ϵ -rule in G' is $S \rightarrow \epsilon$. If G' contains $S \rightarrow \epsilon$ in production rules then $\epsilon \in L(G')$. If G' doesn't contain rule $S \rightarrow \epsilon$ then $\epsilon \notin L(G')$.

T.M T = on input $\langle G, \epsilon \rangle$ where $G = \text{CFG}$

• Convert $G = \text{CFG } G'$

• If G' has the production rule $S \rightarrow \epsilon$ then accept. else $\rightarrow$ reject.

Conclusion: It's clear $\langle G, \epsilon \rangle \in A_{\text{ECFG}}$ iff $\langle G, \epsilon \rangle$ it also belongs to the A_{ECFG} . So above construction is correct. $A_{\text{ECFG}} = \text{Language}$ is decidable